

Independent Spanning Trees on Two-dimension Tori

Shyue-Ming Tang and Yue-Li Wang

All correspondence should be addressed to Professor Yue-Li Wang, Department of Information Management, National Taiwan University of Science and Technology, 43, Section 4, Kee-Lung Road, Taipei, Taiwan, Republic of China (e-mail: ylwang@cs.ntust.edu.tw).

This work was supported by the National Science Council, Republic of China, under Contract 90-2213-E-011-067.

Abstract

Two spanning trees of a given graph G having the same root t are said to be *independent* if for each vertex v of G the two paths from t to v along these two trees are vertex-disjoint. A set of spanning trees of G is said to be independent if they are pairwise independent. A two-dimension *torus* is a mesh with wrap-around connections in both the vertical and horizontal directions. Any two-dimension torus has four independent spanning trees rooted at the same vertex. In this paper, a linear time algorithm is proposed to construct four independent spanning trees on a two-dimension torus.

Keywords: independent spanning trees, vertex-disjoint path, connectivity, regular graph, fault-tolerant broadcasting, torus network.

1. Introduction

A two-dimension *torus*, denoted by $R(r, c)$, is a graph $G = (V, E)$ with $V = \{0, 1, 2, \dots, rc-1\}$ and $E = \{(u, v) \mid v = [u \pm c]_{rc} \text{ or } u - [u]_c + [u \pm 1]_c\}$, where $[u]_c$ denotes u modulo c . If both r and c are greater than two, $R(r, c)$ is 4-connected and 4-regular. We show $R(4, 3)$ in Figure 1.

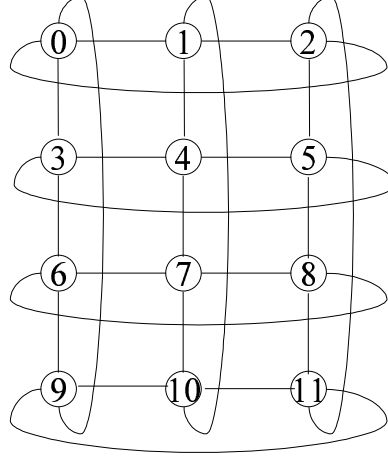


Figure 1. A torus network $R(4, 3)$.

Two-dimension tori (or torus networks) are important due to its simple structure and suitability for developing algorithms [1,4,5]. There are commercially available parallel computers, such as the Intel Paragon, which are equipped with the torus interconnection structure. Therefore, it is valuable to investigate the communication problems on torus networks.

A set of paths connecting two vertices in a graph is said to be *internally disjoint* if and only if any pair of paths in the set has no common vertex except the two end vertices. Considering a graph $G=(V,E)$, a tree T is called a *spanning tree* of G if T is a subgraph of G and T contains all vertices in V . Two spanning trees of G are said to be *independent* if they are rooted at the same vertex, say t , and for each vertex $v \in V \setminus \{t\}$, the two paths from t to v , one path in each tree, are internally disjoint. A set of spanning trees of a graph is said to be independent if they are pairwise independent. For example, four independent spanning trees of $R(4,3)$ are shown in Figure 2.

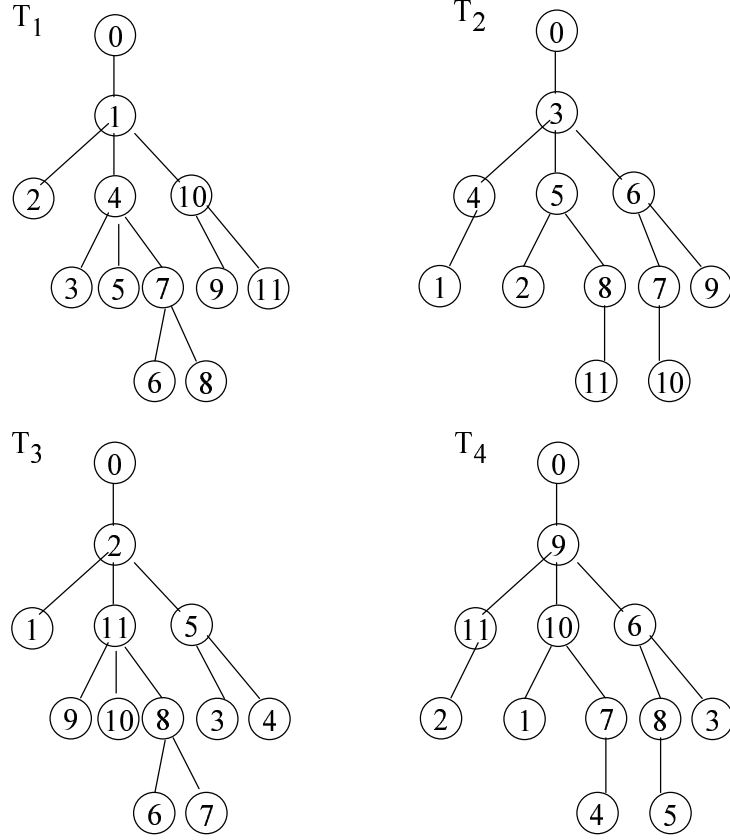


Figure 2. Four independent spanning trees of $R(4, 3)$.

Recently, the problem of finding independent spanning trees of a given graph has received increasing attention. The studies on independent spanning trees find applications in fault-tolerant protocols for distributed computing networks. For example, broadcasting in a network is sending a message from a given node to all other nodes in the network. We can design a fault-tolerant broadcasting scheme based on independent spanning trees [2] [8]. The fault-tolerance can be achieved by sending k copies of the message along k independent spanning trees rooted at the source node. If the source node is faultless, this scheme can tolerate up to $k-1$ faulty nodes.

Itai and Rodeh [8] gave a linear time algorithm for finding two independent spanning trees in a biconnected graph. Cheriyan and Maheshwari [3] showed that, for any 3-connected graph G and for any vertex t of G , three independent spanning trees rooted at t can be found in $O(|V||E|)$ time. In [9] and [11], the authors conjectured that any k -connected graph has k independent spanning trees rooted at an arbitrary vertex r . In 1999, Huck [7] has proved that the conjecture is true for planar graphs. The conjecture is still open for arbitrary k -connected graphs with $k > 3$.

Obokata et al. mentioned [10] that a n -dimension torus has $2n$ independent spanning trees rooted at any vertex. However, they did not give any algorithm to construct these independent spanning trees. In this paper, we shall propose a linear time algorithm to construct four independent spanning trees on a two-dimension torus.

The remaining part of this paper is organized as follows. In Section 2, we propose an algorithm for constructing four independent spanning trees on a two-dimension torus. In Section 3, we discuss the correctness of our algorithm. Section 4 contains our concluding remarks.

2. Construction of independent spanning trees

Torus networks are vertex-symmetric[6]. Without loss of generality, we simply consider independent spanning trees rooted at vertex 0 of a torus network. Let $R(r, c)$ be a two-dimension torus with vertices labeled from 0 to $rc-1$. Four neighbors of vertex v are $[v+c]_{rc}$, $[v-c]_{rc}$, $v-[v]_c+[v+1]_c$ and $v-[v]_c+[v-1]_c$. We propose an algorithm to construct four trees rooted at vertex 0 on $R(r, c)$. Let T_1 , T_2 , T_3 and T_4 denote the trees. The algorithm contains six steps. Each step follows the same rule and builds up a set of edges in T_1 , T_2 , T_3 and T_4 . The edge set created by different steps are defined as *backbone edges*, *side edges*, *neck edge*, *pike edges*, *rib edges* and *hair edges*.

At the first step, we start from vertex 0 to create *backbone edges* for every spanning tree. The backbone edges of T_1 contain $(0,1)$, $(1,2)$, ..., $(c-2,c-1)$. In T_2 , they are $(0,c)$, $(c,2c)$, ..., $((r-2)c,(r-1)c)$. Edges $(0,c-1)$, $(c-1,c-2)$, ..., $(2,1)$ form the backbone edges of T_3 . The backbone edges of T_4 consist of $(0, (r-1)c)$, $((r-1)c,(r-2)c)$, ..., $(2c,c)$. Obviously, the backbone edges of T_1 , T_2 , T_3 and T_4 are developed toward four different directions from vertex 0. Each spanning tree has $c-1$ (in T_1 , T_3) or $r-1$ (in T_2 , T_4) backbone edges. For implementing this step, we have Procedure *Backbone_edges*.

```

Procedure Backbone_edges( $r, c$ )
  ( $T_1$ ) for  $i := 0$  to  $c-2$  do
    connect vertex  $i$  with vertex  $i+1$  ;
  ( $T_2$ ) for  $i := 0$  to  $r-2$  do
    connect vertex  $ci$  with vertex  $c(i+1)$  ;
  ( $T_3$ ) for  $i := 0$  to  $c-2$  do
    connect vertex  $[c-i]_c$  with vertex  $c-(i+1)$  ;
  ( $T_4$ ) for  $i := 0$  to  $r-2$  do

```

connect vertex $[c(r-i)]_{rc}$ with vertex $c(r-(i+1))$

end

At the second step, we extend the backbone edges to create *side edges* for every spanning tree. The side edges of T_1 contain $(1, (r-1)c+1)$, $(2, (r-1)c+2)$, ..., $(c-2, (r-1)c+c-2)$. In T_2 , they are $(c, c+1)$, $(2c, 2c+1)$, ..., $((r-2)c, (r-2)c+1)$. Edges $(c-1, 2c-1)$, $(c-2, 2c-2)$, ..., $(2, c+2)$ form the side edges of T_3 . The side edges of T_4 consist of $((r-1)c, rc-1)$, $((r-2)c, (r-1)c-1)$, ..., $(2c, 3c-1)$. Side edges are vertical to backbone edges. Each spanning tree has $c-2$ (in T_1, T_3) or $r-2$ (in T_2, T_4) side edges. For implementing this step, we have Procedure *Side_edges*.

Procedure *Side_edges*(r, c)

(T_1) **for** $i := 1$ **to** $c-2$ **do**

connect vertex i with vertex $[i-c]_{rc}$;

(T_2) **for** $i := 1$ **to** $r-2$ **do**

connect vertex ci with vertex $ci+1$;

(T_3) **for** $i := 2$ **to** $c-1$ **do**

connect vertex i with vertex $i+c$;

(T_4) **for** $i := 2$ **to** $r-1$ **do**

connect vertex ci with vertex $c(i+1)-1$

end

At the third step, we create a *neck edge* for every spanning tree. It is extended from the first backbone edge. The neck edge of T_1, T_2, T_3 and T_4 is $(1, c+1)$, $(c, 2c-1)$, $(c-1, rc-1)$ and $((r-1)c, (r-1)c+1)$, respectively. Not like other types of edges, there is only one neck edge in every spanning tree. For implementing this edge, we have Procedure *Neck_edge*.

Procedure *Neck_edge*(r, c)

(T_1) connect vertex 1 with vertex $c+1$;

(T_2) connect vertex c with vertex $2c-1$;

(T_3) connect vertex $c-1$ with vertex $rc-1$;

(T_4) connect vertex $(r-1)c$ with vertex $(r-1)c+1$

end

At the fourth step, we extend the neck edge to create *pike edges* for every spanning tree. The pike edges of T_1 contain $(c+1, c+2)$, $(c+2, c+3)$, ...,

$(2c-3, 2c-2)$. In T_2 , they are $(2c-1, 3c-1)$, $(3c-1, 4c-1)$, ..., $((r-2)c-1, (r-1)c-1)$. Edges $(rc-1, rc-2)$, $(rc-2, rc-3)$, ..., $((r-1)c+3, (r-1)c+2)$ form the pick edges of T_3 . The pike edges of T_4 consist of $((r-1)c+1, (r-2)c+1)$, $((r-2)c+1, (r-3)c+1)$, ..., $(3c+1, 2c+1)$. Pike edges are horizontal to backbone edges. Each spanning tree has $c-3$ (in T_1, T_3) or $r-3$ (in T_2, T_4) pike edges. For implementing this step, we have Procedure *Pike_edges*.

```

Procedure Pike_edges( $r, c$ )
  ( $T_1$ ) for  $i := 1$  to  $c-3$  do
    connect vertex  $c+i$  with vertex  $c+i+1$  ;
  ( $T_2$ ) for  $i := 2$  to  $r-2$  do
    connect vertex  $ci-1$  with vertex  $c(i+1)-1$  ;
  ( $T_3$ ) for  $i := 1$  to  $c-3$  do
    connect vertex  $rc-i$  with vertex  $rc-(i+1)$  ;
  ( $T_4$ ) for  $i := 1$  to  $r-3$  do
    connect vertex  $c(r-i)+1$  with vertex  $c(r-(i+1))+1$ 
end

```

At the fifth step, we extend the pike edge to create *rib edges* for every spanning tree. Rib edges are vertical to backbone edges and pike edges. Most edges of the spanning trees are rib edges if r and c are large enough. There are $(r-3)(c-2)$ rib edges in T_1 and T_3 , or $(r-2)(c-3)$ rib edges in T_2 and T_4 . For implementing this step, we have Procedure *Rib_edges*.

```

Procedure Rib_edges( $r, c$ )
  ( $T_1$ ) for  $i := 1$  to  $c-2$  do
    for  $j := 1$  to  $r-3$  do
      connect vertex  $cj+i$  with vertex  $c(j+1)+i$  ;
  ( $T_2$ ) for  $i := 2$  to  $r-1$  do
    for  $j := 1$  to  $c-3$  do
      connect vertex  $ci-j$  with vertex  $ci-(j+1)$  ;
  ( $T_3$ ) for  $i := 1$  to  $c-2$  do
    for  $j := 0$  to  $r-2$  do
      connect vertex  $c(r-j)-i$  with vertex  $c(r-(j+1))-i$  ;
  ( $T_4$ ) for  $i := 2$  to  $r-1$  do
    for  $j := 1$  to  $c-3$  do
      connect vertex  $c(r-i)+j$  with vertex  $c(r-i)+(j+1)$ 
end

```

At the last step, we have to connect all residual vertices with suitable tree edges. Thus *hair edges* are created for every spanning tree. Hair edges are horizontal to backbone edges. There are $2(r-1)$ hair edges in T_1 and T_3 , or $2(c-1)$ hair edges in T_2 and T_4 . For implementing this step, we have Procedure *Hair_edges*.

```

Procedure Hair_edges( $r, c$ )
  ( $T_1$ ) for  $i := 1$  to  $r-1$  do begin
    connect vertex  $ci+1$  with vertex  $ci$  ;
    connect vertex  $c(i+1)-2$  with vertex  $c(i+1)-1$ 
  end ;
  ( $T_2$ ) for  $i := 1$  to  $c-1$  do begin
    connect vertex  $i+c$  with vertex  $i$  ;
    connect vertex  $i+c(r-2)$  with vertex  $i+c(r-1)$ 
  end ;
  ( $T_3$ ) for  $i := 1$  to  $r-1$  do begin
    connect vertex  $c(i+1)-1$  with vertex  $ci$  ;
    connect vertex  $ci+2$  with vertex  $ci+1$ 
  end ;
  ( $T_4$ ) for  $i := 1$  to  $c-1$  do begin
    connect vertex  $[i-c]_{rc}$  with vertex  $i$  ;
    connect vertex  $i+2c$  with vertex  $i+c$ 
  end
end

```

Then, we describe our algorithm for constructing four trees based on a two-dimension torus $R(r, c)$.

Algorithm IST_Torus

Input: $R(r, c)$.

Output: T_1, T_2, T_3 and T_4 .

Method:

Step 1. call *backbone_edges* ;

Step 2. call *side_edges* ;

Step 3. call *neck_edge* ;

Step 4. call *pike_edges* ;

Step 5. call *rib_edges* ;

Step 6. call *hair_edges*
End of Algorithm IST_Torus

For example, we construct T_1, T_2, T_3 and T_4 on $R(5,6)$ as shown in Figure 3. Notice that different types of lines indicate different tree edges.

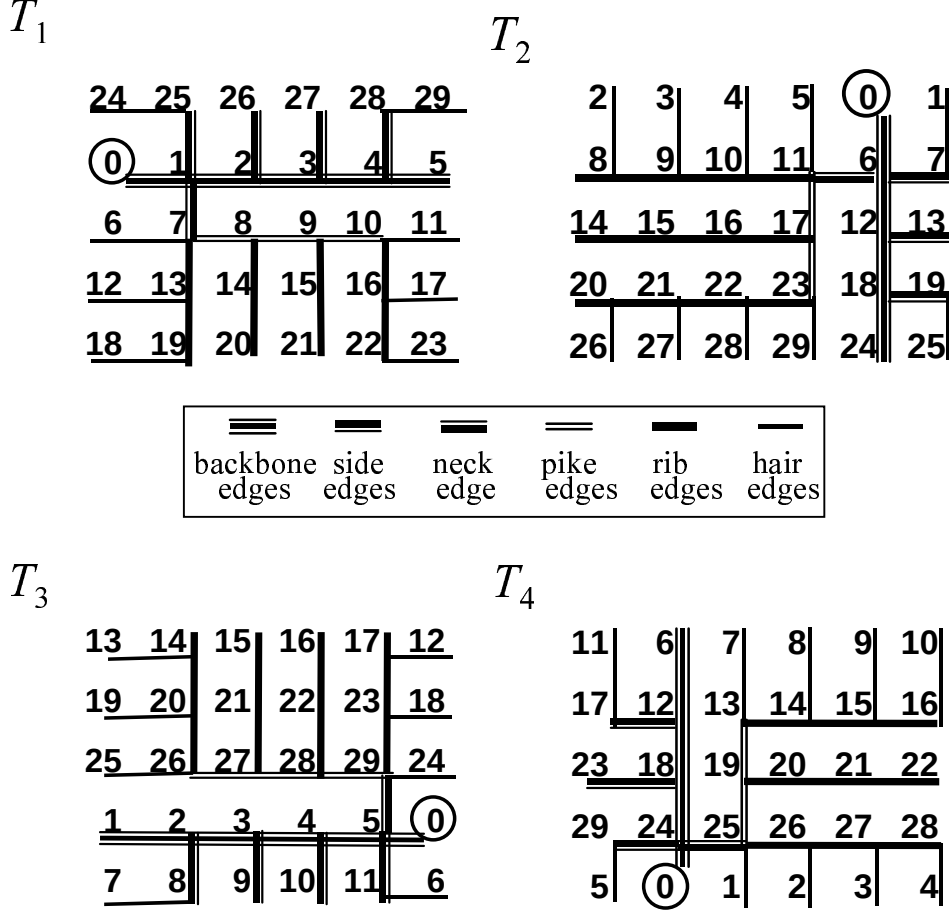


Figure 3. T_1, T_2, T_3 and T_4 of $R(5,6)$

Obviously, the time complexity of **Algorithm IST_Torus** is $O(rc)$ which is bound by **Step 5**.

3. Correctness

In this section, we shall prove that T_1, T_2, T_3 and T_4 are independent spanning trees rooted at 0 on $R(r,c)$.

Lemma 1. T_1, T_2, T_3 and T_4 are spanning trees of $R(r,c)$.

Proof. Based on the steps of **Algorithm IST_Torus**, T_1 , T_2 , T_3 and T_4 are connected trees. Meanwhile, the number of edges in T_1 and T_3 is $c-1 + c-2 + 1 + c-3 + (c-2)(r-3) + 2(r-1) = rc-1$. The number of edges in T_2 and T_4 is $r-1 + r-2 + 1 + r-3 + (c-3)(r-2) + 2(c-1) = rc-1$. Therefore, T_1 , T_2 , T_3 and T_4 are four spanning trees of $R(r, c)$. **Q.E.D.**

Now, we have to prove that T_1 , T_2 , T_3 and T_4 are pairwise independent. For this purpose, we define the *ancestor* of a vertex v in T_i ($i=1,2,3,4$), denoted by $ancestor(v, i)$, is the vertex set of the path from 0 to the *parent* vertex of v in T_i . By the definition of independent spanning trees, we have the proposition: “ T_i and T_j ($i \neq j$) are independent if and only if for every vertex v in $R(r, c)$, $v \neq 0$, $ancestor(v, i) \cap ancestor(v, j) = \{0\}$ ”. This proposition is helpful in proving the following lemma.

Lemma 2. Let v be a vertex in $R(r, c)$ and $1 \leq v \leq c-1$, then

$$\bigcap_{i=1,2,3,4} ancestor(v, i) = \{0\}.$$

Proof. See Figure 4. In case that $v = 1$, $ancestor(1, 1) \cap ancestor(1, 2) \cap ancestor(1, 3) \cap ancestor(1, 4) = \{0\} \cap \{c+1, c, 0\} \cap \{2, 3, \dots, c-1, 0\} \cap \{c(r-1)+1, c(r-1), 0\} = \{0\}$. In case that $1 < v < c-1$, $ancestor(v, 1) \cap ancestor(v, 2) \cap ancestor(v, 3) \cap ancestor(v, 4) = \{v-1, v-2, \dots, 1, 0\} \cap \{v+c, v+c+1, \dots, 2c-1, c, 0\} \cap \{v+1, v+2, \dots, c-1, 0\} \cap \{[v-c]_{rc}, [v-c]_{rc}-1, \dots, c(r-1), 0\} = \{0\}$. In case that $v = c-1$, $ancestor(c-1, 1) \cap ancestor(c-1, 2) \cap ancestor(c-1, 3) \cap ancestor(c-1, 4) = \{c-2, c-3, \dots, 0\} \cap \{2c-1, c, 0\} \cap \{0\} \cap \{rc-1, c(r-1), 0\} = \{0\}$. **Q.E.D.**

$c(r-1)$	$c(r-1)-1$	...	$[v-c]_{rc}-1$	$[v-c]_{rc}$		...	$rc-1$	$c(r-1)$
0	1	...	$v-1$	v	$v+1$	...	$c-1$	0
c	$c+1$	...		$v+c$	$v+c+1$	...	$2c-1$	c

Figure 4. Four paths from 0 to v in $R(r, c)$, $1 \leq v \leq c-1$

Lemma 3. Let v be a vertex in $R(r, c)$ and $c \leq v \leq 2c-1$, then

$$\bigcap_{i=1,2,3,4} ancestor(v, i) = \{0\}.$$

Proof. See Figure 5. In case that $v = c$, $ancestor(c, 1) \cap ancestor(c, 2) \cap ancestor(c, 3) \cap ancestor(c, 4) = \{c+1, 1, 0\} \cap \{0\} \cap \{2c-1, c-1, 0\} \cap \{2c, 3c, \dots, c(r-1), 0\} = \{0\}$. In case that $v = c+1$, $ancestor(c+1, 1) \cap ancestor(c+1, 2) \cap ancestor(c+1, 3) \cap ancestor(c+1, 4) = \{1, 0\} \cap \{c, 0\} \cap \{c+2, 2, \dots, c-1, 0\} \cap \{2c+1, 3c+1, \dots, c(r-1)+1, c(r-1), 0\} = \{0\}$. In case that $c+1 < v < 2c-1$,

$ancestor(v,1) \cap ancestor(v,2) \cap ancestor(v,3) \cap ancestor(v,4) = \{v-1, v-2, \dots, c+1, 1, 0\} \cap \{v+1, v+2, \dots, 2c-1, c, 0\} \cap \{v-c, v-c+1, \dots, c-1, 0\} \cap \{v+c, v+c-1, \dots, 2c+1, \dots, c(r-1)+1, c(r-1), 0\} = \{0\}$. In case that $v = 2c-1$, $ancestor(v,1) \cap ancestor(v,2) \cap ancestor(v,3) \cap ancestor(v,4) = \{v-1, v-2, \dots, c+1, 1, 0\} \cap \{c, 0\} \cap \{3c-1, 2c, \dots, c(r-1), 0\} = \{0\}$. **Q.E.D.**

$3c$	$3c+1$	...	...	...	...	...	$4c-1$	$3c$
...	...	...	...	...	...	...	...	...
$c(r-1)$	$c(r-1)+1$	...	...	...	...	...	$rc-1$	$c(r-1)$
0	1	...	...	$v-c$	$v-c+1$	...	$c-1$	0
c	$c+1$	...	$v-1$	v	$v+1$	...	$2c-1$	c
$2c$	$2c+1$	...	$v+c-1$	$v+c$	...	...	$3c-1$	$2c$

Figure 5. Four paths from 0 to v in $R(r,c)$, $c \leq v \leq 2c-1$

Lemma 4. Let v be a vertex in $R(r,c)$ and $ci \leq v \leq c(i+1)-1$, $2 \leq i \leq r-2$ then

$$\bigcap_{i=1,2,3,4} ancestor(v,i) = \{0\}.$$

Proof. See Figure 6. In case that $v = ci$, $ancestor(v,1) \cap ancestor(v,2) \cap ancestor(v,3) \cap ancestor(v,4) = \{v+1, v+1-c, \dots, 1, 0\} \cap \{v-c, v-2c, \dots, c, 0\} \cap \{v-1, v-1+c, \dots, rc-1, c-1, 0\} \cap \{v+c, v+2c, \dots, c(r-1), 0\} = \{0\}$. In case that $v = ci+1$, $1 < i < r-1$, $ancestor(v,1) \cap ancestor(v,2) \cap ancestor(v,3) \cap ancestor(v,4) = \{v-c, v-2c, \dots, 1, 0\} \cap \{v-1, v-1-c, \dots, c, 0\} \cap \{v+1, v+1+c, \dots, c(r-1)+2, \dots, rc-1, c-1, 0\} \cap \{v+c, v+2c, \dots, c(r-1)+1, c(r-1), 0\} = \{0\}$. In case that $ci+1 < v < c(i+1)-1$, $ancestor(v,1) \cap ancestor(v,2) \cap ancestor(v,3) \cap ancestor(v,4) = \{v-c, v-2c, \dots, [v]_c+c, \dots, c+1, 1, 0\} \cap \{v+1, v+2, \dots, c(i+1)-1, ci-1, \dots, 2c-1, c, 0\} \cap \{v+c, v+2c, \dots, c(r-1)+[v]_c, \dots, rc-1, c-1, 0\} \cap \{v-1, v-2, \dots, ci+1, c(i+1)+1, \dots, c(r-1)+1, c(r-1), 0\} = \{0\}$. In case that $v = c(i+1)-1$, $ancestor(v,1) \cap ancestor(v,2) \cap ancestor(v,3) \cap ancestor(v,4) = \{v-1, v-1-c, v-1-2c, \dots, 2c-2, 2c-3, \dots, c+1, 1, 0\} \cap \{v-c, v-2c, \dots, 2c-1, c, 0\} \cap \{v+c, v+2c, \dots, rc-1, c-1, 0\} \cap \{ci, c(i+1), \dots, c(r-1), 0\} = \{0\}$. **Q.E.D.**

0	1	2	...	$[v]_c-1$	$[v]_c$	$[v]_c+1$	...	$c-2$	$c-1$
c	$c+1$	$c+2$	...	$[v]_c+c-1$	$[v]_c+c$	...	...	$2c-2$	$2c-1$
...	...	...	...	...	...	...	...	...	...
$ci-c$	$ci+1-c$	$ci+2-c$	...	...	$v-c$	...	...	$ci-2$	$ci-1$
ci	$ci+1$	$ci+2$	...	$v-1$	v	$v+1$	...	$c(i+1)-2$	$c(i+1)-1$
$c(i+1)$	$c(i+1)+1$	$c(i+1)+2$	...	...	$v+c$	...	...	$c(i+1)$	$c(i+2)-1$
...	...	...	...	...	...	...	...	...	...
$c(r-1)$	$c(r-1)+1$	$c(r-1)+2$	...	...	$c(r-1)+[v]_c$	...	...	$c(r-1)$	$rc-1$
0	1	2	...	$[v]_c-1$	$[v]_c$	$[v]_c+1$	...	$c-2$	$c-1$

Figure 6. Four paths from 0 to v in $R(r,c)$, $ci \leq v \leq c(i+1)-1$, $2 \leq i \leq r-2$

Lemma 5. Let v be a vertex in $R(r,c)$ and $c(r-1) \leq v \leq rc-1$, then

$$\bigcap_{i=1,2,3,4} \text{ancestor}(v,i) = \{0\}.$$

Proof. See Figure 7. In case that $v = c(r-1)$, $\text{ancestor}(v,1) \cap \text{ancestor}(v,2) \cap \text{ancestor}(v,3) \cap \text{ancestor}(v,4) = \{c(r-1)+1, 1, 0\} \cap \{c(r-2), \dots, 2c, c, 0\} \cap \{rc-1, c-1, 0\} \cap \{0\} = \{0\}$. In case that $v = c(r-1)+1$, $\text{ancestor}(v,1) \cap \text{ancestor}(v,2) \cap \text{ancestor}(v,3) \cap \text{ancestor}(v,4) = \{1, 0\} \cap \{c(r-2)+1, c(r-2), \dots, c, 0\} \cap \{v+1, v+2, \dots, rc-1, c-1, 0\} \cap \{c(r-1), 0\} = \{0\}$. In case that $c(r-1)+1 < v < rc-1$, $\text{ancestor}(v,1) \cap \text{ancestor}(v,2) \cap \text{ancestor}(v,3) \cap \text{ancestor}(v,4) = \{[v+c]_{rc}, [v+c]_{rc}-1, \dots, 1, 0\} \cap \{v-c, v-c+1, \dots, c(r-1)-1, \dots, 2c-1, c, 0\} \cap \{v+1, \dots, rc-1, c-1, 0\} \cap \{v-1, v-2, \dots, c(r-1), 0\} = \{0\}$. In case that $v = rc-1$, $\text{ancestor}(v,1) \cap \text{ancestor}(v,2) \cap \text{ancestor}(v,3) \cap \text{ancestor}(v,4) = \{rc-2, c-2, \dots, 1, 0\} \cap \{c(r-1)-1, \dots, 2c-1, c, 0\} \cap \{c-1, 0\} \cap \{c(r-1), 0\} = \{0\}$. **Q.E.D.**

$c(r-2)$	$c(r-2)+1$	...	...	$v-c$	$v-c+1$	...	$c(r-1)-1$	$c(r-2)$
$c(r-1)$	$c(r-1)+1$	...	$v-1$	v	$v+1$	...	$rc-1$	$c(r-1)$
0	1	...	$[v+c]_{rc}-1$	$[v+c]_{rc}$	...	...	$c-1$	0
c	$c+1$	...	...	...	...	...	$2c-1$	c
...	...	...	...	...	...	...	...	...
$c(r-3)$	$c(r-3)+1$	...	...	...	...	...	$c(r-2)-1$	$c(r-3)$

Figure 7. Four paths from 0 to v in $R(r,c)$, $c(r-1) \leq v \leq rc-1$

Therefore, for every vertex v in $R(r,c)$, $v \neq 0$, $\text{ancestor}(v,1) \cap \text{ancestor}(v,2) \cap \text{ancestor}(v,3) \cap \text{ancestor}(v,4) = \{0\}$. That is, T_1, T_2, T_3 and T_4 are mutually independent.

We summarize Lemmas 1, 2, 3, 4 and 5 as Theorem 6.

Theorem 6. Algorithm *IST_Torus* correctly construct four independent spanning trees rooted at 0 on $R(r,c)$ in $O(rc)$ time.

4. Concluding remarks

In this paper, we present a linear-time algorithm for constructing four independent spanning trees of a two-dimension torus $R(r,c)$. The height of every spanning tree is $r+c-3$. This result is exactly good but not the only solution. It also remains unknown whether this result is the optimal solution. We are going to investigate these problems.

There are many vertex-symmetric graphs, such as hypercubes, star graphs,

recursive circulant graphs, honeycomb tori, and so on. Linear-time algorithms to find independent spanning trees for these classes of graphs are valuable. New skills for verification of the independency are also eagerly needed in developing these algorithms.

References

- [1] Myung M. Bae and Bella Bose, "Resource placement in torus-based networks," *IEEE Transactions on Computers* 46(10), 1997, pp. 1083-1092.
- [2] F. Bao, Y. Igarashi, S.R. Öhring, "Reliable broadcasting in product networks," *IEICE Technical Report COMP* 95(18), 1995, pp.57-66.
- [3] J. Cheriyan, S. N. Maheshwari, "Finding nonseparating induced cycles and independent spanning trees in 3-connected graphs," *Journal of Algorithms* 9, 1988, pp.507-537.
- [4] Christoph von Conta, "Torus and other networks as communication networks with up to some hundred points," *IEEE Transactions on Computers* 32(7), 1983, pp.657-666.
- [5] Robert Cypher and Luis Gravano, "Storage-efficient and, deadlock-free packet routing algorithms for torus networks," *IEEE Transactions on Computers* 43(12), 1994, pp.1376-1385.
- [6] Frank Harary, "Graph Theory," Addison-Wesley, 1968, pp.171-173.
- [7] A. Huck, "Independent trees in planar graphs," *Graphs and Combinatorics* 15, 1999, pp.29-77.
- [8] A. Itai, M. Rodeh, "The multi-tree approach to reliability in distributed networks," *Information and Computation* 79, 1988, pp.43-59.
- [9] Samir Khuller, Baruch Schieber, "On independent spanning trees", *Information Processing Letters* 42, 1992, pp.321-323.
- [10] K. Obokata, Y. Iwasaki, F. Bao and Y. Igarashi, "Independent spanning trees of product graphs," *Lecture Notes in Computer Science* 1197, 1996, pp.338-351.
- [11] A. Zehavi, A. Itai, "Three tree-paths," *Journal of Graph Theory* 13, 1989, pp.175-188.